

02244 Logic for Security

Security Protocols

Automated Analysis: Abstraction

Sebastian Mödersheim

March 16, 2026

Challenges From Last Week

Example/FOO

$$\alpha \equiv x[1], x[2], x[3] \in \{0, 1\} \wedge x[1] + x[2] + x[3] = 1$$

$$\beta \equiv x[\pi[1]] = 1 \wedge x[\pi[2]] = 0 \wedge x[\pi[3]] = 0 \wedge$$

π is a permutation on $\{1, 2, 3\}$

- Consider the model $x[1] = 0, x[2] = 1, x[3] = 0$ of α .
 - ★ Find a permutation $\pi : \{1, 2, 3\} \rightarrow \{1, 2, 3\}$ such that β holds.

Challenges From Last Week

Example/FOO

$$\alpha \equiv x[1], x[2], x[3] \in \{0, 1\} \wedge x[1] + x[2] + x[3] = 1$$

$$\beta \equiv x[\pi[1]] = 1 \wedge x[\pi[2]] = 0 \wedge x[\pi[3]] = 0 \wedge$$

π is a permutation on $\{1, 2, 3\}$

- Consider the model $x[1] = 0, x[2] = 1, x[3] = 0$ of α .
 - ★ Find a permutation $\pi : \{1, 2, 3\} \rightarrow \{1, 2, 3\}$ such that β holds.
 - ★ Solution: either $\pi[1] = 2, \pi[2] = 1, \pi[3] = 3$, or $\pi[1] = 2, \pi[2] = 3, \pi[3] = 1$.

Challenges From Last Week

Example/FOO

$$\begin{aligned}\alpha &\equiv x[1], x[2], x[3] \in \{0, 1\} \wedge x[1] + x[2] + x[3] = 1 \\ \beta &\equiv x[\pi[1]] = 1 \wedge x[\pi[2]] = 0 \wedge x[\pi[3]] = 0 \wedge \\ &\quad \pi \text{ is a permutation on } \{1, 2, 3\}\end{aligned}$$

- Consider the model $x[1] = 0, x[2] = 1, x[3] = 0$ of α .
 - ★ Find a permutation $\pi : \{1, 2, 3\} \rightarrow \{1, 2, 3\}$ such that β holds.
 - ★ Solution: either $\pi[1] = 2, \pi[2] = 1, \pi[3] = 3$, or $\pi[1] = 2, \pi[2] = 3, \pi[3] = 1$.
- Consider the actual messages that C in FOO publishes,

$$\{\text{commit}(r[\pi[j]], v[\pi[j]])\}_{\text{inv}(\text{pk}(A))}, r[\pi[j]]$$

- ★ That is at position j in the publishing, we have the vote of some voter $\pi[j]$.
- ★ Can a voter *prove* to the intruder which one is their vote?
- ★ Is that a desirable property of the protocol?

Challenge from Last Week

$$\{commit(r[\pi[j]], v[\pi[j]])_{inv(pk(A))}, r[\pi[j]]$$

Can a voter *prove* to the intruder which one is their vote?

Solution: when all votes are published and the commitments open, every voter can **claim any vote** to be his/hers.

- Even if the voter gives out their private key it does not tell the intruder which their true vote was.
- For the blinded signed message exchanged with the administrator:
 - ★ To claim a vote $cv = \{commit([r[\pi[k]], v[\pi[k]])]_{inv(pk(A))}$
 - ★ choose blinding value b and tell the intruder $blind(b, cv)$ was what the administrator sent you.
- There is no way that an honest agent can **prove** to the intruder how they voted – it could always be a lie.
- This is called **receipt freeness** and an important tool against coercing voters, and in general for privacy.

Challenge from Last Week

FOO is **not** receipt free in an earlier stage:

- When all then unopened commitments are published, then every voter can tell their actual commitment secret $r[i]$ to the intruder and thereby prove “authorship” of the vote.
- If the administrator is dishonest, it is also not receipt free.

First-Order Logic (FOL)

Syntax

$Formula ::= Predicate(Term, \dots, Term)$
| $Formula \wedge Formula$
| $\neg Formula$
| $\exists Variable. Formula$

Example: $\forall k. \forall m. i(\{m\}_k) \wedge i(k) \implies i(m)$

Syntactic Sugar

$\forall x. \phi \equiv \neg \exists x. \neg \phi$
 $\phi \vee \psi \equiv \neg(\neg \phi \wedge \neg \psi)$
 $\phi \implies \psi \equiv \neg \phi \vee \psi$

First-Order Logic (FOL)

Interpretation

An **interpretation** $\mathcal{I}$ consists of the following:

- A non-empty set D : the **domain** (or **universe**)
- For every function symbol f of arity n have some function $f^{\mathcal{I}} : D^n \rightarrow D$
- For every predicate symbol p of arity n have some relation $p^{\mathcal{I}} \subset D^n$.
- For every variable symbol x , a value from D .

Extend $\mathcal{I}$ to function on terms:

$$\mathcal{I}(f(t_1, \dots, t_n)) = f^{\mathcal{I}}(\mathcal{I}(t_1), \dots, \mathcal{I}(t_n))$$

First-Order Logic (FOL)

We now define what it means that an interpretation $\mathcal{I}$ is a **model** of a formula ϕ (or “ $\mathcal{I}$ satisfies ϕ ”):

Semantics

Let $\mathcal{I}$ be an interpretation over domain D , and ϕ a formula. Define $\mathcal{I} \models \phi$ (read “ $\mathcal{I}$ models ϕ ”) as follows:

$\mathcal{I} \models p(t_1, \dots, t_n)$	iff	$(\mathcal{I}(t_1), \dots, \mathcal{I}(t_n)) \in p^{\mathcal{I}}$
$\mathcal{I} \models \phi \wedge \psi$	iff	$\mathcal{I} \models \phi$ and $\mathcal{I} \models \psi$
$\mathcal{I} \models \neg \phi$	iff	not $\mathcal{I} \models \phi$
$\mathcal{I} \models \exists x. \phi$	iff	there is a $c \in D$ such that $\mathcal{I}[x \mapsto c] \models \phi$

where $\mathcal{I}[x \mapsto c]$ is the same interpretation as $\mathcal{I}$ except that x is mapped to c .

Finally, define $\phi \models \psi$ iff every model of ϕ is also a model of ψ .

First-Order Logic (FOL)

$$\begin{aligned} & (\quad \forall k. \forall m. i(\{m\}_k) \wedge i(k) \implies i(m) \\ & \quad \wedge i(sec) \implies attack \\ & \quad \wedge i(\{sec\}_{pw}) \wedge i(pw) \quad) \\ & \models attack \end{aligned}$$

SPASS

SPASS [Weidenbach et al. 1996ff]

Automatic Theorem Prover for First-Order Logic (FOL)

- Given a formula, SPASS either
 - ★ finds a proof of validity
 - ★ or a counter-example
 - ★ or runs into an infinite loop
 - ▶ Validity for FOL is undecidable

SPASS for Protocol Verification?

We can express Dolev-Yao intruder deduction in FOL
(Intuition: $i(m)$ represents “intruder knows message m ”)

Dolev-Yao in FOL

$$\forall m, k : \quad i(m) \wedge i(k) \implies i(\{m\}_k)$$

$$\forall m, k : \quad i(\{m\}_k) \wedge i(k) \implies i(m)$$

$$\forall m, k : \quad i(m) \wedge i(k) \implies i(\{m\}_k)$$

$$\forall m, k : \quad i(\{m\}_k) \wedge i(\text{inv}(k)) \implies i(m)$$

$$\forall m_1, m_2 : \quad i(\langle m_1, m_2 \rangle) \implies i(m_1) \wedge i(m_2)$$

$$\forall m_1, m_2 : \quad i(m_1) \wedge i(m_2) \implies i(\langle m_1, m_2 \rangle)$$

Let φ be the conjunction of these intruder deduction rules, then

$$\varphi \wedge i(\{secret\}_{pk(i)}) \wedge i(\text{inv}(pk(i))) \implies i(secret)$$

is a theorem that SPASS can automatically prove.

Protocols in SPASS

- Dolev-Yao intruder deduction works just fine, but what about
 - ★ Freshly generated numbers/keys?
 - ★ Any notion of time/progress?
 - ▶ E.g. a number can only be used once.

Protocols in SPASS

- Dolev-Yao intruder deduction works just fine, but what about
 - ★ Freshly generated numbers/keys?
 - ★ Any notion of time/progress?
 - ▶ E.g. a number can only be used once.
- Actually, this is not directly possible in FOL
 - ★ We cannot formalize the concept of reachable states in FOL.
 - ★ This is a limitation of FOL.

Protocols in SPASS

- Dolev-Yao intruder deduction works just fine, but what about
 - ★ Freshly generated numbers/keys?
 - ★ Any notion of time/progress?
 - ▶ E.g. a number can only be used once.
- Actually, this is not directly possible in FOL
 - ★ We cannot formalize the concept of reachable states in FOL.
 - ★ This is a limitation of FOL.
- Actually, limitations can sometimes inspire great ideas!
 - ★ In this case, one of the most successful tools for protocol verification: [ProVerif](#) [Blanchet 2001]
 - ★ We cover today the most basic ideas of this approach:
 - ▶ How to model protocols in FOL
 - ▶ How to analyze such a FOL formula

No Fresh Numbers?

Example: Needham-Schroeder-Lowe (NSL)

Knowledge :

$A : A, B, pk(A), inv(pk(A)), pk(B);$

$B : B, A, pk(B), inv(pk(B)), pk(A);$

Actions :

$A \rightarrow B : \{f1(NA, A)\}_{pk(B)}$

$B \rightarrow A : \{f2(NA, NB, B)\}_{pk(A)}$

$A \rightarrow B : \{f3(NB)\}_{pk(B)}$

NA secret between A, B

NB secret between A, B

No Fresh Numbers?

Example: Needham-Schroeder-Lowe (NSL)

Knowledge :

$A : A, B, pk(A), inv(pk(A)), pk(B);$

$B : B, A, pk(B), inv(pk(B)), pk(A);$

Actions :

$A \rightarrow B : \{f1(NA, A)\}_{pk(B)}$

$B \rightarrow A : \{f2(NA, NB, B)\}_{pk(A)}$

$A \rightarrow B : \{f3(NB)\}_{pk(B)}$

NA secret between A, B

NB secret between A, B

- If we are only interested in secrecy (and weak authentication), but not in **freshness**, actually we would not need that NA and NB are really fresh.

No Fresh Numbers?

Example: Needham-Schroeder-Lowe (NSL)

Knowledge :

$A : A, B, pk(A), inv(pk(A)), pk(B);$

$B : B, A, pk(B), inv(pk(B)), pk(A);$

Actions :

$A \rightarrow B : \{f1(NA, A)\}_{pk(B)}$

$B \rightarrow A : \{f2(NA, NB, B)\}_{pk(A)}$

$A \rightarrow B : \{f3(NB)\}_{pk(B)}$

NA secret between A, B

NB secret between A, B

- If we are only interested in secrecy (and weak authentication), but not in **freshness**, actually we would not need that NA and NB are really fresh.
- It would be actually still fine if A did reuse an older numbers she used with B earlier.

No Fresh Numbers?

Example: Needham-Schroeder-Lowe (NSL)

Knowledge :

$A : A, B, pk(A), inv(pk(A)), pk(B);$

$B : B, A, pk(B), inv(pk(B)), pk(A);$

Actions :

$A \rightarrow B : \{f1(NA, A)\}_{pk(B)}$

$B \rightarrow A : \{f2(NA, NB, B)\}_{pk(A)}$

$A \rightarrow B : \{f3(NB)\}_{pk(B)}$

NA secret between A, B

NB secret between A, B

- If we are only interested in secrecy (and weak authentication), but not in **freshness**, actually we would not need that NA and NB are really fresh.
- It would be actually still fine if A did reuse an older numbers she used with B earlier.
 - ★ It could be a shared secret $na(A, B)$ between A and B .
 - ★ Note: just na or $na(A)$ would lead to attacks – why?

No Fresh Numbers!

Example: NSL without fresh numbers

Knowledge :

$A : A, B, pk(A), inv(pk(A)), pk(B), na(A, B);$

$B : B, A, pk(B), inv(pk(B)), pk(A), nb(B, A);$

Actions :

$A \rightarrow B : \{f1(na(A, B), A)\}_{pk(B)}$

$B \rightarrow A : \{f2(na(A, B), nb(B, A), B)\}_{pk(A)}$

$A \rightarrow B : \{f3(nb(B, A))\}_{pk(B)}$

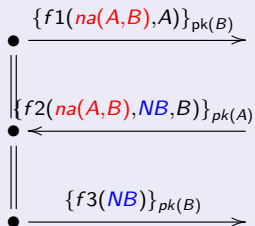
$na(A, B)$ secret between A, B

$nb(B, A)$ secret between A, B

- Note: $na(A, B)$ is initially known only to A , $nb(B, A)$ only to B .
- Idea: if abstract NSL is secure, then the original NSL is secure.

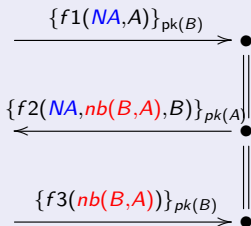
No Fresh Numbers!

Role A: replace NA with $na(A, B)$.



Leave received number NB as is.

Role B: replace NB with $nb(B, A)$.

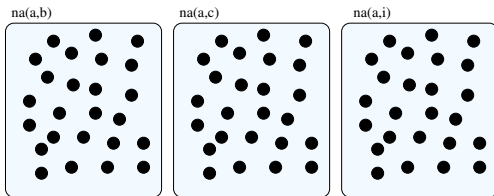


Leave received number NA as is.

- On A 's side only NA is abstracted into $na(A, B)$, because A still accepts any number NB from B .
- Conversely on B 's side.

Abstract Interpretation

We have **partitioned** the set of all **concrete numbers** into **abstract equivalence classes**:

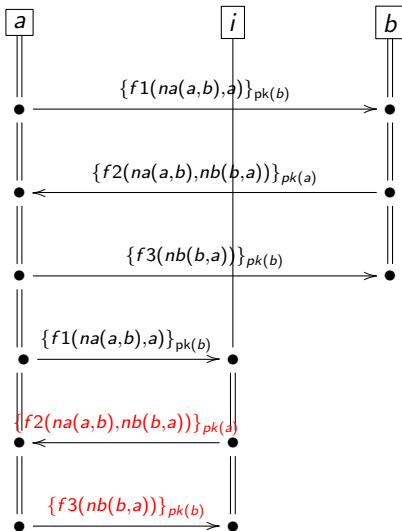


a 's numbers for communication with b , c , and i

and replace in the protocol model each **concrete value** with its **abstract equivalence class**.

Abstract Interpretation

- Every reachable concrete state has an abstract counter-part,
- but some abstract states have no concrete counter-part:



Abstract Interpretation

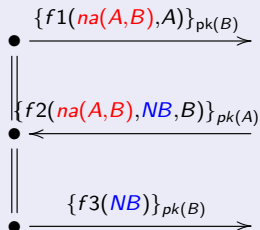
- Every reachable concrete state has an abstract counter-part,
- but some abstract states have no concrete counter-part.
- So the abstract model is an **over-approximation** of the concrete model, allowing at least as many behaviors.
- The abstract model is easier to verify than the concrete one.
- Claim: if the abstract model has no attack trace, then the concrete model has none. (True?)
- An attack trace in the abstract model may be a **false positive**: no (corresponding) attack trace in the concrete model.

Honest Agents in FOL

Suppose we have

- *Hon* set of honest agents ($a, b, ..$)
- *Dis* set of dishonest agents (the intruder $i...$)
- $Agent = Hon \cup Dis$ is the set of all agents
- $\forall A \in Agent : i(A) \wedge i(pk(A))$
- $\forall A \in Dis : i(inv(pk(A)))$
- *Number* be the set of all Numbers ($na(A, B), nb(B, A), ...$)

Honest Agents in FOL

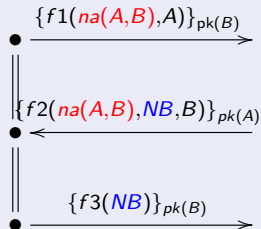


- Role A of NSL
- We only need to describe it for the case $A \in Hon$, because $A \in Dis$ is already covered by the intruder rules.
- For B we must however consider an arbitrary agent.

$$\forall A \in Hon, B \in Agent : \\ i(\{f1(na(A, B), A)\}_{pk(B)})$$

$$\forall A \in Hon, B \in Agent, NB \in Number : \\ i(\{f2(na(A, B), NB, B)\}_{pk(A)}) \implies i(\{f3(NB)\}_{pk(B)})$$

Honest Agents as Oracles

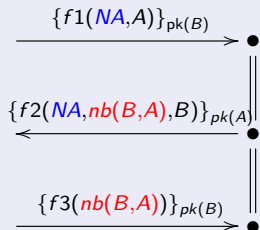


- Since the numbers are not fresh, the intruder can send any number of $f2$ -messages with the **same number** $na(A, B)$ and get an $f3$ -answer.
- A is now just an **oracle** for the intruder.

$$\forall A \in Hon, B \in Agent : \\ i(\{f1(na(A, B), A)\}_{pk(B)})$$

$$\forall A \in Hon, B \in Agent, NB \in Number : \\ i(\{f2(na(A, B), NB, B)\}_{pk(A)}) \implies i(\{f3(NB)\}_{pk(B)})$$

Honest Agents in FOL



- Role B of NSL
- We only need to describe it for the case $B \in Hon$, because $B \in Dis$ is already covered by the intruder rules.
- For A we must however consider an arbitrary agent.

$\forall A \in Agent, B \in Hon, NA \in Number :$

$$i(\{f1(NA, A)\}_{pk(B)}) \implies i(\{f2(NA, nb(B, A), B)\}_{pk(A)})$$

- Also B is now just an oracle for the intruder.
- For secrecy, the step of B receiving $\{f3(nb(B, A))\}_{pk(B)}$ is irrelevant, since the intruder does not learn anything from this.
- For authentication goals, this would have to be modeled.

The Secrecy Goal

The secrecy goal is that the intruder never gets hold of any numbers that were meant for use between **honest agents**:

$$\begin{aligned}\forall A \in Hon, B \in Hon : i(na(A, B)) &\implies \text{attack} \\ \forall A \in Hon, B \in Hon : i(nb(B, A)) &\implies \text{attack}\end{aligned}$$

where *attack* is a new predicate.

It will be our goal to either prove or disprove that *attack* follows from the FOL formula that describes the protocol.

Synopsis

$$\forall m, k : i(m) \wedge i(k) \implies i(\{m\}_k)$$

$$\forall m, k : i(\{m\}_k) \wedge i(k) \implies i(m)$$

$$\forall m, k : i(m) \wedge i(k) \implies i(\{m\}_k)$$

$$\forall m, k : i(\{m\}_k) \wedge i(\text{inv}(k)) \implies i(m)$$

$$\forall m_1, m_2 : i(\langle m_1, m_2 \rangle) \implies i(m_1) \wedge i(m_2)$$

$$\forall m_1, m_2 : i(m_1) \wedge i(m_2) \implies i(\langle m_1, m_2 \rangle)$$

$$\forall A \in \text{Agent} : i(A) \wedge i(\text{pk}(A))$$

$$\forall A \in \text{Dis} : i(\text{inv}(\text{pk}(A)))$$

$$\forall A \in \text{Hon}, B \in \text{Agent} : i(\{f1(\text{na}(A, B), A)\}_{\text{pk}(B)})$$

$$\forall A \in \text{Hon}, B \in \text{Agent}, NB \in \text{Number} :$$

$$i(\{f2(\text{na}(A, B), NB, B)\}_{\text{pk}(A)}) \implies i(\{f3(NB)\}_{\text{pk}(B)})$$

$$\forall A \in \text{Agent}, B \in \text{Hon}, NA \in \text{Number} :$$

$$i(\{f1(NA, A)\}_{\text{pk}(B)}) \implies i(\{f2(NA, \text{nb}(B, A), B)\}_{\text{pk}(A)})$$

$$\forall A \in \text{Hon}, B \in \text{Hon} : i(\text{na}(A, B)) \implies \text{attack}$$

$$\forall A \in \text{Hon}, B \in \text{Hon} : i(\text{nb}(B, A)) \implies \text{attack}$$

Resolution

Resolution

$$\frac{A \implies p \quad p \wedge B \implies q}{A \wedge B \implies q}$$

- During analysis, we typically have a proof state of the form

$$p_1 \wedge p_2 \wedge \dots \wedge p_k \implies \text{attack}$$

so we want to see whether all the p_i can be derived.

- Thus look for any rule $A \implies p_1$:
 - ★ Check out $A \wedge p_2 \wedge \dots \wedge p_k \implies \text{attack}$.
- This is a backward search to prove attack.
- If all such attempts fail, the protocol is **secure**.

Intruder Deduction

It is convenient to handle intruder deduction first:

- For rules that describe the honest agents:
 - ★ See if the intruder can compose the message he sends to them.
 - ★ See if the intruder can decompose/analyze the message he receives from them.
 - ★ In this way, we can **compile in** the Dolev-Yao deduction into the rules, and do not worry about Dolev-Yao during deduction.

Example:

$$\forall A \in Hon, B \in Agent : i(\{f1(na(A, B), A)\}_{pk(B)})$$

allows the intruder to derive $na(A, B)$ if $B \in Dis$:

$$\forall A \in Hon, B \in Dis : i(na(A, B))$$

Synopsis

Intruder Deduction compiled in

$\forall A \in Hon, B \in Agent : i(\{f1(na(A, B), A)\}_{pk(B)})$

$\forall A \in Hon, B \in Dis : i(na(A, B))$

$\forall A \in Agent, B \in Hon, NA \in Number :$

$i(\{f1(NA, A)\}_{pk(B)}) \implies i(\{f2(NA, nb(B, A), B)\}_{pk(A)})$

$\forall A \in Agent, B \in Hon, NA \in Number :$

$i(NA) \implies i(\{f2(NA, nb(B, A), B)\}_{pk(A)})$

$\forall A \in Dis, B \in Hon : i(nb(B, A))$

$\forall A \in Dis, B \in Hon, NA \in Number :$

$i(\{f1(NA, A)\}_{pk(B)}) \implies i(NA) \wedge i(nb(B, A))$

$\forall A \in Hon, B \in Agent, NB \in Number :$

$i(\{f2(na(A, B), NB, B)\}_{pk(A)}) \implies i(\{f3(NB)\}_{pk(B)})$

$\forall A \in Hon, B \in Dis, NB \in Number :$

$i(\{f2(na(A, B), NB, B)\}_{pk(A)}) \implies i(NB)$

$\forall A \in Hon, B \in Hon : i(na(A, B)) \implies \text{attack}$

$\forall A \in Hon, B \in Hon : i(nb(B, A)) \implies \text{attack}$

Search for Attack

To get to an attack there are just two possibilities (renamed A and B to avoid name clashes):

$$\forall A' \in Hon, B' \in Hon : i(na(A', B')) \implies \text{attack}$$

$$\forall A' \in Hon, B' \in Hon : i(nb(B', A')) \implies \text{attack}$$

Any rules that can resolve these numbers:

$$\forall A \in Hon, B \in Dis : i(na(A, B))$$

$$\forall A \in Dis, B \in Hon : i(nb(B, A))$$

$$\begin{aligned} \forall A \in Dis, B \in Hon, NA \in Number : \\ i(\{f1(NA, A)\}_{pk(B)}) \implies i(NA) \wedge i(nb(B, A)) \end{aligned}$$

$$\begin{aligned} \forall A \in Hon, B \in Dis, NB \in Number : \\ i(\{f2(na(A, B), NB, B)\}_{pk(A)}) \implies i(NB) \end{aligned}$$

The dishonest agents narrow our choices here...

Search for Attack

To get to an attack there are just two possibilities (renamed A and B to avoid name clashes):

$$\forall A' \in Hon, B' \in Hon : i(na(A', B')) \implies \text{attack}$$

$$\forall A' \in Hon, B' \in Hon : i(nb(B', A')) \implies \text{attack}$$

Any rules that can resolve these numbers:

$$\forall A \in Dis, B \in Hon, NA \in Number : i(\{f1(NA, A)\}_{pk(B)}) \implies i(NA)$$

$$\forall A \in Hon, B \in Dis, NB \in Number : i(\{f2(na(A, B), NB, B)\}_{pk(A)}) \implies i(NB)$$

This gives us four possibilities:

$$\forall A \in Dis, B, A', B' \in Hon : i(\{f1(na(A', B'), A)\}_{pk(B)}) \implies \text{attack}$$

$$\forall A \in Dis, B, A', B' \in Hon : i(\{f1(nb(B', A'), A)\}_{pk(B)}) \implies \text{attack}$$

$$\forall B \in Dis, A, A', B' \in Hon : i(\{f2(na(A, B), na(A', B'), B)\}_{pk(A)}) \implies \text{attack}$$

$$\forall B \in Dis, A, A', B' \in Hon : i(\{f2(na(A, B), nb(B', A'), B)\}_{pk(A)}) \implies \text{attack}$$

Search for Attack

$\forall A \in Dis, B, A', B' \in Hon : i(\{f1(na(A', B'), A)\}_{pk(B)}) \implies \text{attack}$

$\forall A \in Dis, B, A', B' \in Hon : i(\{f1(nb(B', A'), A)\}_{pk(B)}) \implies \text{attack}$

$\forall B \in Dis, A, A', B' \in Hon : i(\{f2(na(A, B), na(A', B'), B)\}_{pk(A)}) \implies \text{attack}$

$\forall B \in Dis, A, A', B' \in Hon : i(\{f2(na(A, B), nb(B', A'), B)\}_{pk(A)}) \implies \text{attack}$

Rules that generate $f1$ or $f2$ terms:

$\forall A \in Hon, B \in Agent : i(\{f1(na(A, B), A)\}_{pk(B)})$

$\forall A \in Agent, B \in Hon, NA \in Number :$

$i(\{f1(NA, A)\}_{pk(B)}) \implies i(\{f2(NA, nb(B, A), B)\}_{pk(A)})$

$\forall A \in Agent, B \in Hon, NA \in Number :$

$i(NA) \implies i(\{f2(NA, nb(B, A), B)\}_{pk(A)})$

Search for Attack

$$\begin{aligned}\forall A \in Dis, B, A', B' \in Hon : i(\{f1(na(A', B'), A)\}_{pk(B)}) &\implies \text{attack} \\ \forall A \in Dis, B, A', B' \in Hon : i(\{f1(nb(B', A'), A)\}_{pk(B)}) &\implies \text{attack} \\ \forall B \in Dis, A, A', B' \in Hon : i(\{f2(na(A, B), na(A', B'), B)\}_{pk(A)}) &\implies \text{attack} \\ \forall B \in Dis, A, A', B' \in Hon : i(\{f2(na(A, B), nb(B', A'), B)\}_{pk(A)}) &\implies \text{attack}\end{aligned}$$

Rules that generate $f1$ or $f2$ terms:

$$\begin{aligned}\forall A \in Hon, B \in Agent : i(\{f1(na(A, B), A)\}_{pk(B)}) \\ \forall A \in Agent, B \in Hon, NA \in Number : \\ \quad i(\{f1(NA, A)\}_{pk(B)}) \implies i(\{f2(NA, nb(B, A), B)\}_{pk(A)}) \\ \forall A \in Agent, B \in Hon, NA \in Number : \\ \quad i(NA) \implies i(\{f2(NA, nb(B, A), B)\}_{pk(A)})\end{aligned}$$

None of these fit! There is no way to derive attack!

Search for Attack

For the **original** Needham-Schroeder **without Lowe's fix** we have though:

$$\forall B \in Dis, A, A', B' \in Hon : i(\{f2(na(A, B), nb(B', A'))\}_{pk(A)}) \implies \text{attack}$$

and we can solve that with:

$$\forall A' \in Agent, B' \in Hon, NA \in Number : i(NA) \implies i(\{f2(NA, nb(B', A'))\}_{pk(A')})$$

yielding the rule:

$$\forall B \in Dis, A \in Hon : i(na(A, B)) \implies \text{attack}$$

Search for Attack

For the **original** Needham-Schroeder **without Lowe's fix** we have though:

$$\forall B \in Dis, A, A', B' \in Hon : i(\{f2(na(A, B), nb(B', A'))\}_{pk(A)}) \implies \text{attack}$$

and we can solve that with:

$$\forall A' \in Agent, B' \in Hon, NA \in Number : i(NA) \implies i(\{f2(NA, nb(B', A'))\}_{pk(A')})$$

yielding the rule:

$$\forall B \in Dis, A \in Hon : i(na(A, B)) \implies \text{attack}$$

and the intruder indeed knows that $na(A, B)$ for dishonest B , thus we derive **attack**!

Summary: Abstract Interpretation

Abstract Interpretation of Data

Abstract the set of fresh data into finitely many equivalence classes.

Abstraction of States

Instead of a large or infinite state space:

- Fixedpoint of **all facts** that are true **any** reachable state.
- There is no notion of time/progress anymore.

Comparison to chess:

- instead of asking what states of the board are reachable,
- rather ask for each piece what fields of the board it can ever reach.



Over-approximation:

- Not all abstract states are feasible in the real world.
- Works well for simple security protocols,
- but **not for systems with non-monotonic behavior.**

The Logos Case Study



Card Terminal Systems

- Card terminals are produced in large batches.
- Spare terminals reside in storage for up to 10 years.
- Life time of public keys of Logos servers: 2 years.
- How to make a secure connection after 10 years in storage?

The Logos Case Study

Bootstrap Protocol

- Run shortly after manufacturing for every terminal.
- Establish a secure connection to the Logos server (current certificates built in at production)
- Generate a fresh public/private **bootstrap key**
- Public key stored at the Logos server with hardware id.

Endpoint Certificate Protocol

- Run when device comes out of storage.
- Use the **bootstrap key** to establish a secure connection with the Logos server.
- Obtain all current certificates from the Logos server.

Challenges for Verification

- Composition of several protocols
 - ★ Sequential Composition: one protocol establishes keys that another protocol uses.
 - ★ Vertical Composition: the connection between terminal and server is run over TLS.
- Infinite state space: no bound on the number of terminals and sessions.
- Non-monotonicity and long-term state
 - ★ Keys and certificates that are valid now can be invalid later.

Challenges for Verification and Responses

- Composition of several protocols
 - ★ Sequential Composition: one protocol establishes keys that another protocol uses.
 - ★ Vertical Composition: the connection between terminal and server is run over TLS.
- Infinite state space: no bound on the number of terminals and sessions.
- Non-monotonicity and long-term state
 - ★ Keys and certificates that are valid now can be invalid later.

We have tools for that!

Challenges for Verification and Responses

- Composition of several protocols
 - ★ Sequential Composition: one protocol establishes keys that another protocol uses.
 - ★ Vertical Composition: the connection between terminal and server is run over TLS.
 - ★ **Compositional reasoning as seen in week 5.**
- Infinite state space: no bound on the number of terminals and sessions.
- Non-monotonicity and long-term state
 - ★ Keys and certificates that are valid now can be invalid later.

We have tools for that!

Challenges for Verification and Responses

- Composition of several protocols
 - ★ Sequential Composition: one protocol establishes keys that another protocol uses.
 - ★ Vertical Composition: the connection between terminal and server is run over TLS.
 - ★ Compositional reasoning as seen in week 5.
- Infinite state space: no bound on the number of terminals and sessions.
 - ★ Abstract interpretation as seen today
- Non-monotonicity and long-term state
 - ★ Keys and certificates that are valid now can be invalid later.

We have tools for that!

Challenges for Verification and Responses

- Composition of several protocols
 - ★ Sequential Composition: one protocol establishes keys that another protocol uses.
 - ★ Vertical Composition: the connection between terminal and server is run over TLS.
 - ★ Compositional reasoning as seen in week 5.
- Infinite state space: no bound on the number of terminals and sessions.
 - ★ Abstract interpretation as seen today
- Non-monotonicity and long-term state
 - ★ Keys and certificates that are valid now can be invalid later.
 - ★ Stateful abstraction as seen next.

We have tools for that!

Stateful Abstractions

Idea

Modify the data abstraction to include **a bit of state information**

- just enough to make many stateful protocols work
- but without destroying the advantage of abstraction

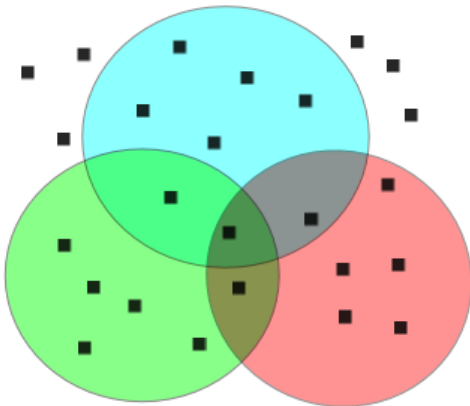
Set-Based Abstraction

Used in the tool **PSPSP** [Hess, M., Brucker, Schlichtkrull]:

- Agents can maintain sets of values (keys, numbers, ...)
 - ★ Adding and deleting elements possible
- **All values are abstracted by their set memberships**
- Updating set membership requires special care...

Set-Based Abstraction

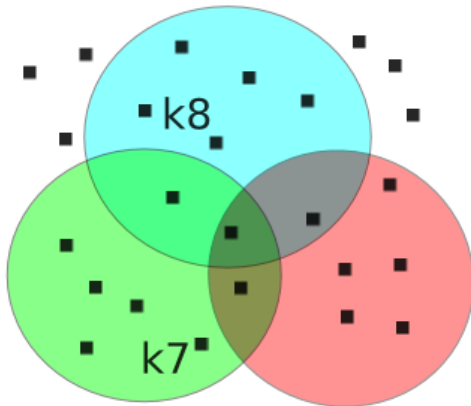
Example: Alice has a set of keys in her **ring** that can be registered at a key server as **valid** or as **revoked**.



Set-Based Abstraction

Concrete intruder knowledge example:

$i(k_7) \cdot i(\text{sign}(\text{inv}(k_7), \text{alice}, k_8)) \cdot i(k_8)$.

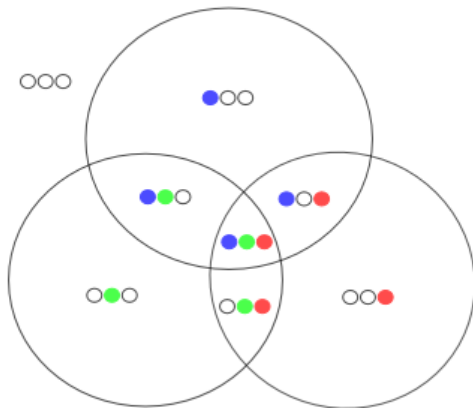


Set-Based Abstraction

Abstract data by their set-membership

Abstract intruder knowledge example:

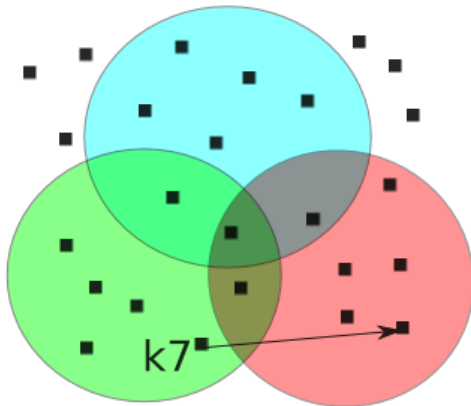
$i(\langle 0, 1, 0 \rangle) \cdot i(\text{sign}(\text{inv}(\langle 0, 1, 0 \rangle), \text{alice}, \langle 1, 0, 0 \rangle)) \cdot i(\langle 1, 0, 0 \rangle).$



Set-Based Abstraction

Concrete data may change its set-membership

Example: Server revokes k_7 when receiving the update.



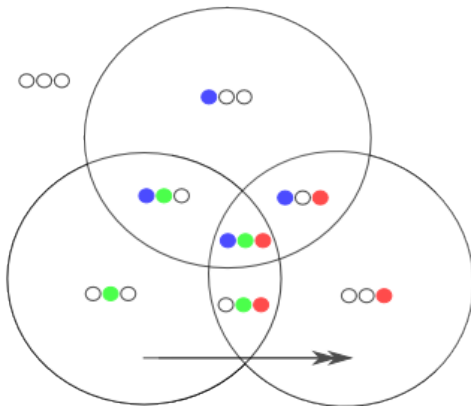
Set-Based Abstraction

How to model the **change of abstraction class**?

Term Implication Rule: $s \twoheadrightarrow t$

$s \twoheadrightarrow t$ means $C[s] \rightarrow C[t]$ for every context C .

“Every fact that is true about s is **also true about t .”**

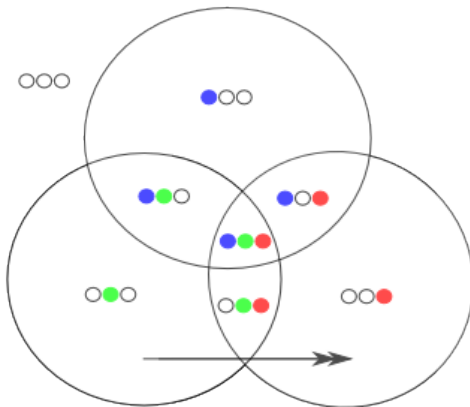


Set-Based Abstraction

Example: $\langle 0, 1, 0 \rangle \twoheadrightarrow \langle 0, 0, 1 \rangle$

given $i(\langle 0, 1, 0 \rangle) \cdot i(\text{sign}(\text{inv}(\langle 0, 1, 0 \rangle), \text{alice}, \langle 1, 0, 0 \rangle)) \cdot i(\langle 1, 0, 0 \rangle).$

then: $i(\langle 0, 0, 1 \rangle) \cdot i(\text{sign}(\text{inv}(\langle 0, 0, 1 \rangle), \text{alice}, \langle 1, 0, 0 \rangle))$



An Attack

The fixedpoint for the Logos protocols contains the symbol **attack**:

The attack

An Attack

The fixedpoint for the Logos protocols contains the symbol **attack**:

The attack

- ① The intruder has access to the device in storage and runs the certificate endpoint protocol, recording all exchanged messages.

An Attack

The fixedpoint for the Logos protocols contains the symbol **attack**:

The attack

- ① The intruder has access to the device in storage and runs the certificate endpoint protocol, recording all exchanged messages.
- ② At any later point, the intruder can **replay** these messages to the device.

An Attack

The fixedpoint for the Logos protocols contains the symbol **attack**:

The attack

- ① The intruder has access to the device in storage and runs the certificate endpoint protocol, recording all exchanged messages.
- ② At any later point, the intruder can **replay** these messages to the device.
- ③ The terminal accepts the certificates from step 1.

An Attack

The fixedpoint for the Logos protocols contains the symbol **attack**:

The attack

- ① The intruder has access to the device in storage and runs the certificate endpoint protocol, recording all exchanged messages.
- ② At any later point, the intruder can **replay** these messages to the device.
- ③ The terminal accepts the certificates from step 1.
- ④ This violates the security goal of certificate authentication.

An Attack

The fixedpoint for the Logos protocols contains the symbol **attack**:

The attack

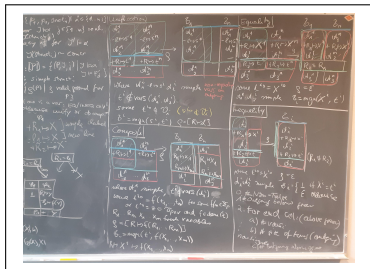
- ① The intruder has access to the device in storage and runs the certificate endpoint protocol, recording all exchanged messages.
- ② At any later point, the intruder can **replay** these messages to the device.
- ③ The terminal accepts the certificates from step 1.
- ④ This violates the security goal of certificate authentication.

The fix

- At the certificate protocol, the device creates a fresh random number that the server includes in its certificate message.
- With that, the Logos protocol verify in PSPSP.
- Logos has implemented this fix and we have published this in our paper on PSPSP.

Conclusions

- Abstract Interpretation is powerful for verification of security protocols
 - ★ side step halting problem and state explosion
- The standard abstraction (e.g. ProVerif) throws away all state information. Too coarse for non-monotonic behaviour.
- Finer abstraction that includes a bit of state:
 - ★ allow for automated verification of non-monotonic systems
 - ★ without killing the spirit of the abstraction.
- Practically useful in industry, e.g.,
 - ★ Industriefondens S3bD
 - ★ EU-Project TaRDIS



A. Hess, S. Mödersheim,
A. Brucker, A. Schlichtkrull:
*PSPSP: A Tool for Automated
Verification of Stateful Proto-
cols in Isabelle/HOL*,
JCS 2025.

Relevant Research Papers

- Christoph Weidenbach: *Towards an Automatic Analysis of Security Protocols in First-Order Logic*, CADE-16, 1999.
- Bruno Blanchet: *An Efficient Cryptographic Protocol Verifier Based on Prolog Rules*, CSFW-14, 2001
- Martin Abadi and Bruno Blanchet: *Analyzing security protocols with secrecy types and logic programs*, POPL 2002.
- Bruno Blanchet and Andreas Podelski: *Verification of Cryptographic Protocols: Tagging Enforces Termination*, FOSSACS 2003.
- Christoph Weidenbach, Dilyana Dimova, Arnaud Fietzke, Rohit Kumar, Martin Suda, and Patrick Wischniewski: *SPASS Version 3.5*, CADE-22, 2009.

Recommended Textbooks on Logic:

- Michael Huth and Mark Ryan: *Logic in computer science*, 2nd edition, Cambridge University Press, 2004.
- Heinz-Dieter Ebbinghaus and Jörg Flum and Wolfgang Thomas: *Mathematical logic*, 2nd edition, Springer, 1994.